합의 성능 최적화

개요

CometBFT의 합의 성능은 네트워크 처리량, 지연시간, 확장성에 직접적인 영향을 미칩니다. 이 문서는 합의 과정에서의 성능 병목 지점을 분석하고, 다양한 최적화 기법과 설정 방법을 제시합니다. 성능 최적화는 네트워크 환경, 하드웨어 사양, 애플리케이션 특성에 따라 달라질 수 있으므로, 각 환경에 맞는 튜닝이 필요합니다.

성능 분석 및 병목 지점

1. 합의 과정의 타이밍 분석

```
gantt

title 합의 과정 타이밍 분석

dateFormat X

axisFormat %s

section Round 0

Propose :0, 3000

Prevote :3000, 4000

Precommit :4000, 5000

Commit :5000, 5500

section Round 1 (timeout)

Propose :6000, 9500
```

```
Prevote :9500, 10500

Precommit :10500, 11500

Next Round :11500, 12000
```

2. 주요 병목 지점

2.1 네트워크 지연

- **문제**: 밸리데이터 간 메시지 전파 지연
- **영향**: 합의 완료 시간 증가, 타임아웃 발생 가능성 증가
- **측정**: 네트워크 RTT, 메시지 전파 시간

2.2 서명 검증

- **문제**: 대량의 투표 서명 검증으로 인한 CPU 부하
- **영향**: 투표 처리 지연, 메모리 사용량 증가
- **측정**: 서명 검증 시간, CPU 사용률

2.3 블록 검증

- **문제**: 큰 블록의 검증 시간
- **영향**: 제안 단계 지연, 전체 합의 시간 증가
- **측정**: 블록 검증 시간, 트랜잭션 처리 시간

타임아웃 최적화

1. 기본 타임아웃 설정

```go
// 기본 합의 타임아웃 설정
type ConsensusConfig struct {
// 제안 대기 시간 (기본: 3초)
TimeoutPropose time.Duration `mapstructure:"timeout_propose"`
// 라운드마다 제안 타임아웃 증가량 (기본: 500ms)
TimeoutProposeDelta time.Duration `mapstructure:"timeout_propose_delta"`
// 투표 대기 시간 (기본: 1초)
TimeoutVote time.Duration `mapstructure:"timeout_vote"`
// 라운드마다 투표 타임아웃 증가량 (기본: 500ms)
TimeoutVoteDelta time.Duration `mapstructure:"timeout_vote_delta"`
// 커밋 후 대기 시간 (deprecated, ABCI에서 설정)
TimeoutCommit time.Duration `mapstructure:"timeout_commit"`
}
```

2. 적응적 타임아웃 계산

```go
// 라운드에 따른 타임아웃 계산
func (cfg *ConsensusConfig) Propose(round int32) time.Duration {
return timeoutTime(cfg.TimeoutPropose, cfg.TimeoutProposeDelta, round)
}

func (cfg *ConsensusConfig) Prevote(round int32) time.Duration {
return timeoutTime(cfg.TimeoutVote, cfg.TimeoutVoteDelta, round)
```

```go
}

func (cfg *ConsensusConfig) Precommit(round int32) time.Duration {
return timeoutTime(cfg.TimeoutVote, cfg.TimeoutVoteDelta, round)
}

func timeoutTime(base, delta time.Duration, round int32) time.Duration {
// 라운드가 증가할수록 타임아웃도 증가
return base + time.Duration(round)*delta
}
```

3. 네트워크 환경별 타임아웃 튜닝

3.1 고속 네트워크 환경

```toml
[consensus]
timeout_propose = "1s"
timeout_propose_delta = "200ms"
timeout_vote = "500ms"
timeout_vote_delta = "200ms"
```

3.2 일반 네트워크 환경

```toml
[consensus]
```

```
timeout_propose = "3s"

timeout_propose_delta = "500ms"

timeout_vote = "1s"

timeout_vote_delta = "500ms"
```

3.3 저속/불안정 네트워크 환경

```
[consensus]

timeout_propose = "10s"

timeout_propose_delta = "2s"

timeout_vote = "3s"

timeout_vote_delta = "1s"
```

메시지 전파 최적화

1. 가십 프로토콜 최적화

```
// P2P 가십 설정

type ConsensusConfig struct {

// 피어 간 가십 슬립 시간 (기본: 100ms)

PeerGossipSleepDuration time.Duration `mapstructure:"peer_gossip_sleep_duration"

// 가십 루프 내 슬립 시간 (기본: 0)

PeerGossipIntraloopSleepDuration time.Duration `mapstructure:"peer_gossip_intral

// Maj23 쿼리 슬립 시간 (기본: 2초)
```

```
PeerQueryMaj23SleepDuration time.Duration `mapstructure:"peer_query_maj23_sleep_
}
```

2. 블록 파트 전송 최적화

```
func (conR *Reactor) gossipDataRoutine(peer p2p.Peer, ps *PeerState) {
// 블록 파트 전송 최적화
for {
rs := conR.getRoundState()
prs := ps.GetRoundState()
// 블록 파트 선택 및 전송
if  part, continueLoop := pickPartToSend(conR.conS.blockStore, &rs, ps, prs, rng
// 병렬 전송으로 지연시간 최소화
if  ps.SendPartSetHasPart(part, prs) || continueLoop {
continue
}
}
// 적응적 슬립 시간
sleepDuration := conR.conS.config.PeerGossipSleepDuration
if  conR.conS.config.PeerGossipIntraloopSleepDuration > 0 {
randDuration := rng.Int63n(int64(conR.conS.config.PeerGossipIntraloopSleepDurati
time.Sleep(time.Duration(randDuration))
```

```
}
time.Sleep(sleepDuration)
}
}
```

3. 투표 전파 최적화

```
func (conR *Reactor) gossipVotesRoutine(peer p2p.Peer, ps *PeerState) {
// 투표 전파 최적화
for {
rs := conR.getRoundState()
prs := ps.GetRoundState()
// 중복 투표 필터링
if  vote := pickVoteToSend(conR.conS, &rs, ps, prs, rng);  vote != nil {
if  ps.sendVoteSetHasVote(vote) {
continue
}
}
// 배치 전송으로 효율성 향상
time.Sleep(conR.conS.config.PeerGossipSleepDuration)
}
}
```

메모리 및 CPU 최적화

1. 투표 집계 최적화

```
// 비트 배열을 사용한 빠른 투표 집계
type VoteSet struct {
votes []*types.Vote
votesBitArray *bits.BitArray // 빠른 비트 연산
sum  int64  // 총 투표권 캐싱
maj23 *types.BlockID // 2/3 이상 지지 블록 캐싱
}

func (voteSet *VoteSet) AddVote(vote *types.Vote) (bool, error) {
// 중복 투표 빠른 확인
if  voteSet.votesBitArray.GetIndex(int(vote.ValidatorIndex)) {
return false, ErrDuplicateVote
}
// 투표 추가 및 비트 설정
voteSet.votes[vote.ValidatorIndex] = vote
voteSet.votesBitArray.SetIndex(int(vote.ValidatorIndex), true)
// 투표권 합계 업데이트
validator := voteSet.valSet.GetByIndex(vote.ValidatorIndex)
voteSet.sum += validator.VotingPower
```

```
// 2/3 이상 확인 (캐싱된 값 사용)
if voteSet.sum > voteSet.valSet.TotalVotingPower()*2/3 {
voteSet.maj23 = &vote.BlockID
}
return true, nil
}
```

2. 서명 검증 최적화

```
// 배치 서명 검증
func (cs *State) batchVerifyVotes(votes []*types.Vote) error {
// 서명 검증을 병렬로 처리
errCh := make(chan error, len(votes))
for _, vote := range votes {
go func(v *types.Vote) {
validator := cs.Validators.GetByIndex(v.ValidatorIndex)
if validator == nil {
errCh <- ErrInvalidValidatorIndex
return
}
// 서명 검증
signBytes := v.SignBytes(cs.state.ChainID)
if !validator.PubKey.VerifySignature(signBytes, v.Signature) {
```

```
errCh <- ErrInvalidSignature
return
}
errCh <- nil
}(vote)
}
// 결과 수집
for i := 0; i < len(votes); i++ {
if err := <-errCh; err != nil {
return err
}
}
return nil
}
```

3. 메모리 풀 최적화

```
// 메모리 사용량 최적화
func (cs *State) optimizeMemoryUsage() {
// 오래된 투표 정리
if cs.Height > cs.lastCleanupHeight+100 {
cs.cleanupOldVotes()
cs.lastCleanupHeight = cs.Height
```

```
}

// 블록 파트 캐시 정리

if  cs.ProposalBlockParts != nil && cs.ProposalBlockParts.IsComplete() {

// 완성된 블록은 압축 저장

cs.compressCompletedBlock()

}

}



func (cs *State) cleanupOldVotes() {

// 현재 높이보다 낮은 투표들 정리

for  height := range  cs.oldVotes {

if  height < cs.Height-1 {

delete(cs.oldVotes, height)

}

}

}
```

네트워크 최적화

1. 연결 관리 최적화

```
[p2p]

# 최대 인바운드 피어 수
```

```
max_num_inbound_peers = 40


# 최대 아웃바운드 피어 수
max_num_outbound_peers = 10


# 연결 플러시 타임아웃
flush_throttle_timeout = "100ms"


# 최대 패킷 크기
max_packet_msg_payload_size = 1024


# 전송 속도 제한
send_rate = 5120000 # 5MB/s
recv_rate = 5120000 # 5MB/s
```

2. 메시지 우선순위 최적화

```
// 메시지 채널별 우선순위 설정
func (conR *Reactor) StreamDescriptors() []p2p.StreamDescriptor {
return []p2p.StreamDescriptor{
// 상태 메시지 (높은 우선순위)
{
ID: StateChannel,
```

```
Priority: 6,
SendQueueCapacity: 100,
RecvMessageCapacity: maxMsgSize,
},
// 데이터 메시지 (최고 우선순위)
{
ID: DataChannel,
Priority: 10,
SendQueueCapacity: 100,
RecvBufferCapacity: 50 * 4096,
RecvMessageCapacity: maxMsgSize,
},
// 투표 메시지 (높은 우선순위)
{
ID: VoteChannel,
Priority: 7,
SendQueueCapacity: 100,
RecvBufferCapacity: 100 * 100,
RecvMessageCapacity: maxMsgSize,
},
// 투표 비트 메시지 (낮은 우선순위)
{
ID: VoteSetBitsChannel,
```

```
Priority: 1,
SendQueueCapacity: 2,
RecvBufferCapacity: 1024,
RecvMessageCapacity: maxMsgSize,
},
}
}
```

3. 대역폭 최적화

```
// 메시지 압축
func (conR *Reactor) compressMessage(msg Message) ([]byte, error) {
// 큰 메시지는 압축하여 전송
msgBytes, err := msg.Marshal()
if err != nil {
return nil, err
}
if len(msgBytes) > 1024 { // 1KB 이상인 경우 압축
var buf bytes.Buffer
writer := gzip.NewWriter(&buf)
if _, err := writer.Write(msgBytes); err != nil {
return nil, err
}
```

```go
if  err := writer.Close();  err != nil {
return  nil, err
}
return  buf.Bytes(), nil
}
return  msgBytes, nil
}
```

블록 크기 및 처리량 최적화

1. 블록 크기 튜닝

```
[consensus]
# 빈 블록 생성 여부
create_empty_blocks = true

# 빈 블록 생성 간격
create_empty_blocks_interval = "0s"

# 최대 블록 크기 (바이트)
max_block_size_bytes = 4194304 # 4MB

# 최대 가스 한도
```

```
max_gas = -1 # 무제한
```

2. 트랜잭션 처리 최적화

```
// 트랜잭션 배치 처리
func (cs *State) optimizeTxProcessing() {
// 트랜잭션을 배치로 처리하여 성능 향상
const  batchSize = 1000
txs := cs.mempool.ReapMaxTxs(batchSize)
if  len(txs) == 0 {
return
}
// 병렬 트랜잭션 검증
validTxs := cs.validateTxsBatch(txs)
// 블록에 포함할 트랜잭션 선택
selectedTxs := cs.selectOptimalTxs(validTxs)
return  selectedTxs
}

func (cs *State) validateTxsBatch(txs []types.Tx) []types.Tx {
validTxs := make([]types.Tx, 0, len(txs))
// 워커 풀을 사용한 병렬 검증
numWorkers := runtime.NumCPU()
```

```go
txCh := make(chan types.Tx, len(txs))
resultCh := make(chan types.Tx, len(txs))
// 워커 시작
for i := 0; i < numWorkers; i++ {
go func() {
for tx := range txCh {
if cs.mempool.CheckTx(tx) == nil {
resultCh <- tx
}
}
}()
}
// 트랜잭션 전송
for _, tx := range txs {
txCh <- tx
}
close(txCh)
// 결과 수집
for i := 0; i < len(txs); i++ {
select {
case validTx := <-resultCh:
validTxs = append(validTxs, validTx)
case <-time.After(100 * time.Millisecond):
```

```
break
}
}
return  validTxs
}
```

모니터링 및 메트릭

1. 성능 메트릭 수집

```
// 합의 성능 메트릭
type Metrics struct {
// 라운드 시간 측정
RoundDuration metrics.Histogram
// 단계별 시간 측정
StepDuration metrics.Histogram
// 투표 처리 시간
VoteProcessingTime metrics.Histogram
// 블록 검증 시간
BlockValidationTime metrics.Histogram
// 네트워크 지연시간
NetworkLatency metrics.Histogram
// 메모리 사용량
```

```
MemoryUsage metrics.Gauge

// CPU 사용률

CPUUsage metrics.Gauge

}



func (cs *State) recordMetrics() {

// 라운드 시간 기록

roundStart := cs.RoundStartTime

if !roundStart.IsZero() {

duration := time.Since(roundStart)

cs.metrics.RoundDuration.Observe(duration.Seconds())

}

// 메모리 사용량 기록

var  m runtime.MemStats

runtime.ReadMemStats(&m)

cs.metrics.MemoryUsage.Set(float64(m.Alloc))

// CPU 사용률 기록 (간단한 예시)

cs.metrics.CPUUsage.Set(cs.getCurrentCPUUsage())

}
```

2. 성능 알림 시스템

```
// 성능 임계값 모니터링
```

```go
func (cs *State) checkPerformanceThresholds() {
// 라운드 시간이 너무 긴 경우
if cs.currentRoundDuration > 30*time.Second {
cs.Logger.Warn("Long consensus round detected",
"duration", cs.currentRoundDuration,
"height", cs.Height,
"round", cs.Round,
)
// 성능 이벤트 발행
cs.eventBus.PublishEventPerformanceWarning(types.EventDataPerformanceWarning{
Type: "long_round",
Duration: cs.currentRoundDuration,
Height: cs.Height,
Round: cs.Round,
})
}
// 메모리 사용량이 높은 경우
var m runtime.MemStats
runtime.ReadMemStats(&m)
if m.Alloc > 1024*1024*1024 { // 1GB
cs.Logger.Warn("High memory usage detected",
"alloc", m.Alloc,
"sys", m.Sys,
```

```
)

// 가비지 컬렉션 강제 실행

runtime.GC()

}

}
```

하드웨어 최적화

1. CPU 최적화

```
// CPU 코어 수에 따른 동적 조정

func (cs *State) optimizeForCPU() {

numCPU := runtime.NumCPU()

// 서명 검증 워커 수 조정

cs.signatureWorkers = numCPU * 2

// 블록 검증 병렬도 조정

cs.blockValidationWorkers = numCPU

// 가비지 컬렉션 튜닝

if  numCPU >= 8 {

debug.SetGCPercent(200) // 고성능 서버

} else {

debug.SetGCPercent(100) // 일반 서버

}
```

```
}
```

2. 메모리 최적화

```
// 메모리 풀 사전 할당
func (cs *State) preallocateMemory() {
// 투표 슬라이스 사전 할당
expectedVotes := cs.Validators.Size() * 3  // Prevote, Precommit, LastCommit
cs.votePool = make([]*types.Vote, 0, expectedVotes)
// 블록 파트 버퍼 사전 할당
cs.blockPartBuffer = make([]byte, 0, 4*1024*1024) // 4MB
// 메시지 버퍼 사전 할당
cs.messageBuffer = make([]Message, 0, 1000)
}
```

3. 디스크 I/O 최적화

```
[consensus]
# WAL 동기화 간격 (성능 vs 안전성 트레이드오프)
wal_file_sync_interval = "1s"


# WAL 파일 크기 제한
wal_file_max_size_bytes = 67108864 # 64MB
```

```
[db]

# 데이터베이스 백엔드 선택

backend = "goleveldb" # 또는 "rocksdb", "badgerdb"



# 캐시 크기

cache_size = 268435456 # 256MB
```

환경별 최적화 가이드

1. 개발 환경

```
[consensus]

timeout_propose = "1s"

timeout_propose_delta = "100ms"

timeout_vote = "500ms"

timeout_vote_delta = "100ms"

create_empty_blocks = true

create_empty_blocks_interval = "1s"



[p2p]

max_num_inbound_peers = 10

max_num_outbound_peers = 5
```

2. 테스트넷 환경

```
[consensus]

timeout_propose = "3s"

timeout_propose_delta = "500ms"

timeout_vote = "1s"

timeout_vote_delta = "500ms"

create_empty_blocks = true

create_empty_blocks_interval = "5s"


[p2p]

max_num_inbound_peers = 20

max_num_outbound_peers = 10
```

3. 메인넷 환경

```
[consensus]

timeout_propose = "5s"

timeout_propose_delta = "1s"

timeout_vote = "2s"

timeout_vote_delta = "1s"

create_empty_blocks = false

create_empty_blocks_interval = "0s"
```

```
[p2p]

max_num_inbound_peers = 40

max_num_outbound_peers = 10
```

성능 테스트 및 벤치마킹

1. 부하 테스트

```
// 합의 성능 벤치마크

func BenchmarkConsensusRound(b *testing.B) {

// 테스트 환경 설정

validators := make([]*types.Validator, 100)

for i := range validators {

validators[i] = types.NewValidator(ed25519.GenPrivKey().PubKey(), 1)

}

cs := newConsensusState(validators)

b.ResetTimer()

for i := 0; i < b.N; i++ {

// 합의 라운드 실행

startTime := time.Now()

cs.runConsensusRound()

duration := time.Since(startTime)
```

```go
// 성능 메트릭 기록
b.ReportMetric(duration.Seconds(), "round_duration")
}
}
```

2. 메모리 프로파일링

```go
// 메모리 사용량 프로파일링
func (cs *State) profileMemory() {
if cs.config.EnableProfiling {
f, err := os.Create("consensus_mem.prof")
if err != nil {
cs.Logger.Error("Could not create memory profile", "err", err)
return
}
defer f.Close()
runtime.GC()
if err := pprof.WriteHeapProfile(f); err != nil {
cs.Logger.Error("Could not write memory profile", "err", err)
}
}
}
```

참고 자료

- CometBFT 성능 가이드
- Tendermint 성능 분석 논문
- 블록체인 합의 성능 최적화

이전 문서: 03-비잔틴-장애-처리.md

다음 문서: .../03-ABCI/01-ABCI-개요.md